

LAPORAN WHITE BOX TESTING — DATA FLOW TESTING

Aplikasi BABEPUS (Barang Bekas Puspita)

1. Pendahuluan & Tujuan Pengujian

Pengujian ini dilakukan untuk menganalisis dan memastikan validitas aliran data dari input pengguna, proses internal server, penyimpanan pada basis data MySQL, hingga output yang dihasilkan oleh aplikasi BABEPUS — sebuah platform marketplace jual beli barang bekas antar mahasiswa.

1.1 Konsep Data Flow Testing

Data Flow Testing adalah metode White Box Testing yang memetakan siklus hidup variabel dalam program. Konsep utamanya adalah:

- Definition (d): Variabel dideklarasikan atau diberi nilai — dari req.body, req.params, req.user, atau hasil await service.
- C-use (Computation Use): Variabel digunakan sebagai argumen fungsi, parameter query, atau nilai response JSON.
- P-use (Predicate Use): Variabel digunakan dalam kondisi percabangan if/ternary yang menentukan alur eksekusi.

1.2 Arsitektur Aliran Data BABEPUS

1. Pengguna mengirim request HTTP dari React.js (frontend) dengan JWT Bearer Token di header.
2. authMiddleware memverifikasi token dan mengisi req.user dengan data user dari database.
3. Controller menerima request dan mendelegasikan ke Service Layer.
4. Service menjalankan logika bisnis dan query database menggunakan parameterized query (pool.query dengan placeholder ?).
5. Database MySQL menyimpan, memperbarui, atau mengembalikan data.
6. Service mengembalikan data ke Controller yang membungkusnya dalam response JSON.

2. Daftar Fitur yang Diuji

Berikut adalah 13 modul controller yang diuji beserta fungsi-fungsi yang dianalisis aliran datanya:

No	Modul	File Controller	Fungsi yang Diuji	Metode HTTP	Status
1	Authentication	authController.js	register(), login(), me(), requestEmailVerification(), verifyEmail()	GET, POST, PUT, DELETE	Valid
2	Admin	adminController.js	getDashboard(),	GET, POST,	Valid

No	Modul	File Controller	Fungsi yang Diuji	Metode HTTP	Status
			getUsers(), suspendUser(), getProducts(), getReports(), updateReportStatus()	PUT, DELETE	
3	Category	categoryController.js	getCategories()	GET	Valid
4	Chat	chatController.js	getConversations(), getMessages(), startConversation(), sendMessage(), streamChat()	GET, POST, PUT, DELETE	Valid
5	Notification	notificationController.js	getNotifications(), markAsRead(), markAllAsRead(), streamNotifications()	GET, POST, PUT, DELETE	Valid
6	Offer	offerController.js	createOffer(), getIncomingOffers(), getMyOffers(), acceptOffer(), rejectOffer()	GET, POST, PUT, DELETE	Valid
7	Pricing	pricingController.js	estimate()	POST	Valid
8	Product	productController.js	listProducts(), searchProducts(), getProduct(), getMyProducts(), createProduct(), updateProduct(), deleteProduct(), markProductSold()	GET, POST, PUT, DELETE	Valid
9	Report	reportController.js	createReport()	POST	Valid
10	Review	reviewController.js	createReview()	POST	Valid
11	Transaction	transactionController.js	getMyTransactions(), completeTransaction(), confirmBuyer(), confirmSeller(), disputeEscrow()	GET, POST, PUT, DELETE	Valid
12	User	userController.js	getDashboardSummary(), updateProfile(), getSellerAnalytics(), uploadAvatar()	GET, POST, PUT, DELETE	Valid
13	Wishlist	wishlistController.js	getWishlist(), addToWishlist(), removeFromWishlist()	GET, POST, PUT, DELETE	Valid

3. Analisis Data Flow per Modul

3.1. Modul Authentication (authController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
register()	req.body (nama, email, password, kampus, NIM)	Hash password (bcrypt), INSERT users dalam DB transaction	201 + token JWT + data user
login()	req.body (email, password)	SELECT user WHERE email, comparePassword(), signToken()	200 + token JWT + data user
me()	req.user.id (dari JWT middleware)	findUserById(), SELECT user dari DB	200 + data profil user
requestEmailVerification()	req.user.id	Cek status verifikasi, generate token verifikasi baru	200 + status verifikasi
verifyEmail()	req.body.token	Validasi token verifikasi, UPDATE isEmailVerified = true	200 + data user terverifikasi

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
email	req.body.email	pool.query WHERE email = ?	C-use	Parameter query SELECT untuk pencarian user di database
password	req.body.password	comparePassword(password, row.passwordHash)	C-use	Argumen fungsi verifikasi bcrypt terhadap hash di DB
rows	await pool.query(SELECT ...)	rows.length, rows[0].isSuspended, rows[0].passwordHash	P-use, C-use	P-use: cek user ada; C-use: ambil hash password dan status suspend
isPasswordValid	await comparePassword()	if (!isPasswordValid)	P-use	Kondisi percabangan login berhasil atau gagal

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
token	signToken({ sub: user.id, role })	return { token, user }	C-use	Disertakan dalam response sebagai JWT bearer token
data (register)	await authService.register(req.body)	res.status(201).json({ data })	C-use	Hasil register langsung dikembalikan sebagai response JSON

Contoh Source Code Aktual:

```
// authController.js - login
const login = asyncHandler(async (req, res) => {
  const data = await authService.login(req.body); // (d) data, C-use: req.body
  res.json({ success: true, data }); // C-use: data
});

// authService.js - login (logic)
const [rows] = await pool.query( // (d) rows
  `SELECT ... FROM users WHERE email = ?`, [email] // C-use: email
);
if (!rows.length) throw new ApiError(401, ...); // P-use: rows
const isPasswordValid = await comparePassword( // (d) isPasswordValid
  password, rows[0].passwordHash // C-use: password, rows
);
if (!isPasswordValid) throw new ApiError(401, ...); // P-use: isPasswordValid
const token = signToken({ sub: user.id }); // (d) token, C-use: user
return { token, user }; // C-use: token, user
```

3.2. Modul Admin (adminController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
getDashboard()	-	getDashboardStats() — query agregat statistik platform	200 + objek stats
getUsers()	req.query (filter/paginasi)	getUsers(req.query) — SELECT users dengan filter	200 + array users
suspendUser()	req.params.id, req.body.isSuspended	suspendUser() — UPDATE isSuspended pada tabel users	200 + data user yang diupdate
getProducts()	-	getProducts() — SELECT semua produk untuk admin	200 + array products
getReports()	-	getReports() — SELECT	200 + array reports

Fungsi	Data Input	Proses Sistem	Output
		semua laporan masuk	
updateReportStatus()	req.params.id, req.body	updateReportStatus() — UPDATE status laporan	200 + data report terupdate

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
stats	await adminService.getDashboardStats()	res.json({ data: { stats } })	C-use	Objek statistik dikembalikan langsung ke response
users	await adminService.getUsers(req.query)	res.json({ data: { users } })	C-use	Array user dikirim sebagai data response
user (suspend)	await adminService.suspendUser(..)	res.json({ data: { user } })	C-use	Objek user terupdate dikembalikan sebagai konfirmasi
req.params.id	URL parameter /:id	adminService.suspendUser(req.user.id, req.params.id, ...)	C-use	ID target user dioper ke service untuk UPDATE
req.body.isSuspended	req.body.isSuspended	suspendUser(..., req.body.isSuspended) dan ternary message	C-use, P-use	Nilai boolean dipakai sebagai argumen dan kondisi pesan response
report	await adminService.updateReportStatus(...)	res.json({ data: { report } })	C-use	Objek report terupdate dikembalikan sebagai response

Contoh Source Code Aktual:

```
// adminController.js - suspendUser
const suspendUser = asyncHandler(async (req, res) => {
  const user = await adminService.suspendUser( // (d) user
    req.user.id, // C-use: req.user.id
    req.params.id, // C-use: req.params.id
    req.body.isSuspended // C-use: req.body.isSuspended
  );
  res.json({
    message: req.body.isSuspended // P-use: req.body.isSuspended
      ? 'User berhasil disuspend.'
      : 'Suspend user berhasil dibuka.',
    data: { user } // C-use: user
  });
});
```

3.3. Modul Category (categoryController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
getCategories()	-	getCategories() — SELECT semua kategori dari tabel categories	200 + array categories

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
categories	await categoryService.getCategories()	res.json({ data: { categories } })	C-use	Array kategori dikirim langsung sebagai data response

Contoh Source Code Aktual:

```
// categoryController.js - getCategories
const getCategories = asyncHandler(async (_req, res) => {
  const categories = await categoryService.getCategories(); // (d) categories
  res.json({ success: true, data: { categories } }); // C-use: categories
});
```

3.4. Modul Chat (chatController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
--------	------------	---------------	--------

Fungsi	Data Input	Proses Sistem	Output
getConversations()	req.user.id	getConversations() — SELECT conversations milik user	200 + array conversations
getMessages()	req.params.id, req.user.id	getMessages() — SELECT pesan dalam conversation	200 + array messages
startConversation()	req.user.id, req.body	startConversation() — INSERT conversation baru atau ambil yang ada	201 + data conversation
sendMessage()	req.params.id, req.user.id, req.body.message	sendMessage() — INSERT pesan, emit SSE event ke lawan chat	201 + objek message
streamChat()	req.query.token (JWT)	getStreamUser(), daftar listener SSE event chat:userId, keepAlive ping 25s	SSE stream (text/event-stream)

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
conversations	await chatService.getConversations(req.user.id)	res.json({ data: { conversations } })	C-use	Daftar percakapan user dikembalikan sebagai response
messages	await chatService.getMessages(req.params.id, req.user.id)	res.json({ data: { messages } })	C-use	Daftar pesan dalam conversation dikembalikan
message (send)	await chatService.sendMessage(..., req.body.message)	res.status(201).json({ data: { message } })	C-use	Objek pesan yang tersimpan dikembalikan
user (stream)	await chatService.getStreamUser(req.query.token)	chatService.chatEvents.on(`chat:\${user.id}`, send)	C-use	ID user dari token dipakai untuk subscribe SSE event
send (fn)	const send = (message) => { res.write(...) }	chatService.chatEvents.on(..., send) dan .off(..., send)	C-use	Fungsi callback

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
				SSE didaftarkan dan dihapus saat koneksi tutup
keepAlive	setInterval(() => res.write(ping), 25000)	clearInterval(keepAlive) saat req.on('close')	C-use	Timer ping SSE dibersihkan saat client disconnect

Contoh Source Code Aktual:

```
// chatController.js - streamChat (SSE pattern)
const streamChat = asyncHandler(async (req, res) => {
  const user = await chatService.getStreamUser(req.query.token); // (d) user
  res.setHeader('Content-Type', 'text/event-stream');
  const send = (message) => { // (d) send
    res.write(`event: message\n`);
    res.write(`data: ${JSON.stringify(message)}\n\n`);
  };
  const keepAlive = setInterval(() => { // (d) keepAlive
    res.write(`event: ping\n`);
  }, 25000);
  chatService.chatEvents.on(`chat:${user.id}`, send); // C-use: user, send
  req.on('close', () => {
    clearInterval(keepAlive); // C-use: keepAlive
    chatService.chatEvents.off(`chat:${user.id}`, send); // C-use: user, send
  });
});
```

3.5. Modul Notification (notificationController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
getNotifications()	req.user.id	getNotifications() — SELECT notifikasi user beserta hitungan unread	200 + array notifications + jumlah unread
markAsRead()	req.params.id, req.user.id	markAsRead() — UPDATE is_read = true WHERE id dan user_id	200 + objek notification terupdate
markAllAsRead()	req.user.id	markAllAsRead() — UPDATE is_read = true WHERE user_id	200 + jumlah baris terupdate
streamNotifications()	req.query.token	getStreamUser(), daftar	SSE stream

Fungsi	Data Input	Proses Sistem	Output
	(JWT)	listener SSE notification:userId, ping 25s	(text/event-stream)

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
data (getNotif)	await notificationService.getNotifications(req.user.id)	res.json({ data })	C-use	Objek berisi array notifications dan unread count dikembalikan
notification (markRead)	await notificationService.markAsRead(req.params.id, req.user.id)	res.json({ data: { notification } })	C-use	Notifikasi yang sudah diupdate dikembalikan sebagai konfirmasi
user (stream)	await notificationService.getStreamUser(req.query.token)	notificationService.notificationEvents.on(`notification:\${user.id}`, send)	C-use	ID user dari token dipakai untuk subscribe SSE event notifikasi
send (fn)	const send = (notification) => { res.write(...) }	.on(..., send) dan .off(..., send)	C-use	Callback SSE didaftarkan dan dibersihkan saat koneksi tutup
keepAlive	setInterval(() => res.write(ping), 25000)	clearInterval(keepAlive) saat req.on('close')	C-use	Timer ping SSE dibersihkan saat

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
				client disconnect

Contoh Source Code Aktual:

```
// notificationController.js - markAsRead
const markAsRead = asyncHandler(async (req, res) => {
  const notification = await notificationService.markAsRead( // (d) notification
    req.params.id,    // C-use: req.params.id
    req.user.id      // C-use: req.user.id
  );
  res.json({ success: true, data: { notification } }); // C-use: notification
});
```

3.6. Modul Offer (offerController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
createOffer()	req.user.id, req.body (productId, harga tawar)	createOffer() — validasi produk, INSERT offer, kirim notifikasi ke penjual	201 + objek offer
getIncomingOffers()	req.user.id	getIncomingOffers() — SELECT offer masuk pada produk milik user	200 + array offers
getMyOffers()	req.user.id	getMyOffers() — SELECT offer yang dikirim oleh user	200 + array offers
acceptOffer()	req.params.id (offerId), req.user.id	acceptOffer() — UPDATE status offer = accepted, auto- CREATE transaction	200 + objek offer + objek transaction
rejectOffer()	req.params.id (offerId), req.user.id	rejectOffer() — UPDATE status offer = rejected	200 + objek offer

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
offer (create)	await offerService.createOffer(req.user.id, req.body)	res.status(201).json({ data: { offer } })	C- use	Objek offer yang tersimpan dikembalikan

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
				n sebagai response
offers (incoming)	await offerService.getIncomingOffers(req.user.id)	res.json({ data: { offers } })	C-use	Array penawaran masuk dikembalikan ke penjual
offers (my)	await offerService.getMyOffers(req.user.id)	res.json({ data: { offers } })	C-use	Array penawaran yang dikirim user dikembalikan
data (accept)	await offerService.acceptOffer(req.params.id, req.user.id)	res.json({ data })	C-use	Berisi offer + transaction yang otomatis dibuat
offer (reject)	await offerService.rejectOffer(req.params.id, req.user.id)	res.json({ data: { offer } })	C-use	Offer yang ditolak dikembalikan sebagai konfirmasi

Contoh Source Code Aktual:

```
// offerController.js - acceptOffer
const acceptOffer = asyncHandler(async (req, res) => {
  const data = await offerService.acceptOffer( // (d) data
    req.params.id, // C-use: req.params.id (offerId)
    req.user.id   // C-use: req.user.id
  );
  res.json({
    message: 'Tawaran diterima dan transaksi otomatis dibuat.',
    data     // C-use: data (berisi offer + transaction)
  });
});
```

3.7. Modul Pricing (pricingController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
estimate()	req.body (kategori, kondisi, harga beli, tahun)	estimateUsedPrice() — kalkulasi estimasi harga barang bekas berdasarkan faktor depresiasi	200 + objek estimasi harga

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
estimateResult	await pricingService.estimateUsedPrice(req.body)	res.json({ data: { estimate: estimateResult } })	C-use	Hasil kalkulasi estimasi harga dikembalikan sebagai response

Contoh Source Code Aktual:

```
// pricingController.js - estimate
const estimate = asyncHandler(async (req, res) => {
  const estimateResult = await pricingService.estimateUsedPrice(req.body); // (d) estimateResult
  res.json({ success: true, data: { estimate: estimateResult } }); // C-use: estimateResult
});
```

3.8. Modul Product (productController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
listProducts()	req.query (filter, paginasi), req.user?.id	getProducts() — SELECT produk aktif dengan filter kategori, harga, kondisi; paginasi	200 + array products + meta paginasi
searchProducts()	req.query.search / req.query.q	getProducts() dengan parameter search — LIKE pada title & description	200 + array products + meta
getProduct()	req.params.id, req.user?.id	getProductById() — SELECT produk lengkap + wishlist status	200 + objek product detail
getMyProducts()	req.user.id	getMyProducts() — SELECT produk milik user yang login	200 + array products
createProduct()	req.user.id, req.body, req.file	Validasi file upload, getUploadUrl(), createProduct() — INSERT produk baru	201 + objek product
updateProduct()	req.params.id, req.user, req.body, req.file	getUploadUrl() jika ada file, updateProduct() — cek ownership, UPDATE produk	200 + objek product terupdate
deleteProduct()	req.params.id,	deleteProduct() — cek	200 + objek product

Fungsi	Data Input	Proses Sistem	Output
	req.user	ownership, soft-delete (deleted_at = NOW())	terhapus
markProductSold()	req.params.id, req.user	markProductSold() — cek ownership, UPDATE status = sold	200 + objek product

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
currentUserId	req.user?.id null	productService.getProducts(req.query, currentUserId)	C-use	Optional chaining; null jika tidak login, ID jika sudah login
data (list)	await productService.getProducts(...)	products: data.products, meta: data.meta	C-use	Objek dengan array produk dan meta paginasi di-destructure untuk response
imageUrl (create)	getUploadUrl(req.file.filename)	productService.createProduct(req.user.id, req.body, imageUrl)	C-use	URL file upload dioper ke service untuk disimpan ke kolom image_url
imageUrl (update)	req.file ? getUploadUrl(req.file.filename) : null	productService.updateProduct(..., imageUrl)	C-use	Null jika tidak ada file baru; URL jika ada file baru diunggah
product (create)	await productService.createProduct(...)	res.status(201).json({ data: { product } })	C-use	Produk yang tersimpan dikembalikan sebagai konfirmasi

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
product (delete)	await productService.deleteProduct(req.params.id, req.user)	res.json({ data: { product } })	C-use	Produk yang di-soft-delete dikembalikan sebagai konfirmasi

Contoh Source Code Aktual:

```
// productController.js - createProduct
const createProduct = asyncHandler(async (req, res) => {
  if (!req.file) throw new ApiError(422, 'Gambar produk wajib diunggah.');// P-use: req.file
  const product = await productService.createProduct( // (d) product
    req.user.id, // C-use: req.user.id
    req.body, // C-use: req.body
    getUploadUrl(req.file.filename) // C-use: req.file.filename
  );
  res.status(201).json({ data: { product } }); // C-use: product
});

// productController.js - updateProduct
const updateProduct = asyncHandler(async (req, res) => {
  const imageUrl = req.file ? getUploadUrl(req.file.filename) : null; // (d) imageUrl
  const product = await productService.updateProduct( // (d) product
    req.params.id, req.user, req.body, imageUrl // C-use: imageUrl
  );
  res.json({ data: { product } }); // C-use: product
});
```

3.9. Modul Report (reportController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
createReport()	req.user.id, req.body (targetId, targetType, alasan)	createReport() — INSERT laporan, notifikasi ke admin	201 + objek report

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
report	await reportService.createReport(req.user.id,	res.status(201).json({ data: { report } })	C-use	Laporan yang

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
	req.body)			tersimpan dikembalikan sebagai konfirmasi pengiriman

Contoh Source Code Aktual:

```
// reportController.js - createReport
const createReport = asyncHandler(async (req, res) => {
  const report = await reportService.createReport( // (d) report
    req.user.id, // C-use: req.user.id
    req.body // C-use: req.body
  );
  res.status(201).json({ data: { report } }); // C-use: report
});
```

3.10. Modul Review (reviewController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
createReview()	req.user.id, req.body (transactionId, rating, komentar)	createReview() — validasi transaksi selesai, INSERT review, UPDATE rating rata-rata penjual	201 + objek review

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
review	await reviewService.createReview(req.user.id, req.body)	res.status(201).json({ data : { review } })	C-use	Objek review yang tersimpan dikembalikan sebagai response

Contoh Source Code Aktual:

```
// reviewController.js - createReview
const createReview = asyncHandler(async (req, res) => {
  const review = await reviewService.createReview( // (d) review
    req.user.id, // C-use: req.user.id
    req.body // C-use: req.body (transactionId, rating, komentar)
  );
  res.status(201).json({ data: { review } }); // C-use: review
});
```

3.11. Modul Transaction (transactionController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
getMyTransactions()	req.user.id	getMyTransactions() — SELECT transaksi sebagai pembeli atau penjual	200 + array transactions
completeTransaction()	req.params.id, req.user.id	completeTransaction() — validasi kedua pihak konfirmasi, UPDATE status = completed	200 + objek transaction
confirmBuyer()	req.params.id, req.user.id	confirmEscrow(..., 'buyer') — UPDATE buyer_confirmed = true	200 + objek transaction
confirmSeller()	req.params.id, req.user.id	confirmEscrow(..., 'seller') — UPDATE seller_confirmed = true	200 + objek transaction
disputeEscrow()	req.params.id, req.user.id, req.body.note	disputeEscrow() — INSERT dispute record, UPDATE status = disputed, notifikasi admin	200 + objek transaction

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
transactions	await transactionService.getMyTransactions(req.user.id)	res.json({ data: { transactions } })	C-use	Array transaksi (sebagai penjual dan pembeli) dikembalikan
transaction (complete)	await transactionService.completeTransaction(req.params.id, req.user.id)	res.json({ data: { transaction } })	C-use	Transaksi yang diselesaikan dikembalikan sebagai konfirmasi
transaction	await	res.json({ data:	C-	Transaksi

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
n (confirm)	transactionService.confirmEscrow(req.params.id, req.user.id, 'buyer'/'seller')	{ transaction } }	use	terupdate setelah konfirmasi escrow dikembalikan
transaction (dispute)	await transactionService.disputeEscrow(req.params.id, req.user.id, req.body.note)	res.json({ data: { transaction } })	C-use	Transaksi dengan status disputed dikembalikan
req.body.note	req.body.note	transactionService.disputeEscrow(..., req.body.note)	C-use	Catatan dispute dioper ke service untuk disimpan ke DB

Contoh Source Code Aktual:

```
// transactionController.js - disputeEscrow
const disputeEscrow = asyncHandler(async (req, res) => {
  const transaction = await transactionService.disputeEscrow( // (d) transaction
    req.params.id, // C-use: req.params.id
    req.user.id, // C-use: req.user.id
    req.body.note // C-use: req.body.note
  );
  res.json({ data: { transaction } }); // C-use: transaction
});
```

3.12. Modul User (userController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
getDashboardSummary()	req.user.id	getDashboardSummary() — query agregat penjualan, transaksi, review user	200 + objek data summary
updateProfile()	req.user.id, req.body (nama, bio, dsb.)	updateProfile() — UPDATE kolom profil pada tabel users	200 + objek user terupdate
getSellerAnalytics()	req.user.id	getSellerAnalytics() — query statistik produk, penjualan, rating penjual	200 + objek analytics
uploadAvatar()	req.user.id,	Validasi file ada,	200 + objek user

Fungsi	Data Input	Proses Sistem	Output
	req.file	getUploadUrl(), updateAvatar() — UPDATE avatar_url di DB	terupdate

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
data (dashboard)	await userService.getDashboardSummary(req.user.id)	res.json({ data })	C-use	Objek ringkasan dashboard dikembalikan langsung
user (profile)	await userService.updateProfile(req.user.id, req.body)	res.json({ data: { user } })	C-use	Profil user yang terupdate dikembalikan
analytics	await userService.getSellerAnalytics(req.user.id)	res.json({ data: { analytics } })	C-use	Objek analitik penjual dikembalikan sebagai response
user (avatar)	await userService.updateAvatar(req.user.id, getUploadUrl(req.file.filename))	res.json({ data: { user } })	C-use	User dengan avatar URL baru dikembalikan
req.file (check)	req.file dari multer middleware	if (!req.file) throw new ApiError(422, ...)	P-use	P-use: kondisi pengecekan apakah file avatar diunggah

Contoh Source Code Aktual:

```
// userController.js - uploadAvatar
const uploadAvatar = asyncHandler(async (req, res) => {
  if (!req.file) throw new ApiError(422, 'Avatar wajib diunggah.');// P-use: req.file
  const user = await userService.updateAvatar( // (d) user
    req.user.id, // C-use: req.user.id
    getUploadUrl(req.file.filename) // C-use: req.file.filename
  );
  res.json({ data: { user } }); // C-use: user
});
```

3.13. Modul Wishlist (wishlistController.js)

Tabel Aliran Data:

Fungsi	Data Input	Proses Sistem	Output
getWishlist()	req.user.id	getWishlist() — SELECT produk dalam wishlist user (JOIN products)	200 + array products
addToWishlist()	req.user.id, req.params.productId	addToWishlist() — INSERT ke tabel wishlists (cek duplikat)	201 + objek wishlist
removeFromWishlist()	req.user.id, req.params.productId	removeFromWishlist() — DELETE dari tabel wishlists WHERE user_id dan product_id	200 + konfirmasi

Tabel DU-Pairs (Definition-Use):

Variabel	Definition (d)	Use (u)	Jenis Use	Keterangan
products (wishlist)	await wishlistService.getWishlist(req.user.id)	res.json({ data: { products } })	C-use	Array produk dalam wishlist user dikembalikan
wishlist (add)	await wishlistService.addToWishlist(req.user.id, req.params.productId)	res.status(201).json({ data: { wishlist } })	C-use	Record wishlist yang baru ditambahkan dikembalikan
wishlist (remove)	await wishlistService.removeFromWishlist(req.user.id, req.params.productId)	res.json({ data: { wishlist } })	C-use	Konfirmasi penghapusan dari wishlist dikembalikan
req.params.productId	URL parameter /:productId	wishlistService.addToWishlist(..., req.params.productId) dan removeFromWishlist(..., req.params.productId)	C-use	ID produk dari URL dioper ke service untuk operasi INSERT/DELETE

Contoh Source Code Aktual:

```
// wishlistController.js - addToWishlist
const addToWishlist = asyncHandler(async (req, res) => {
  const wishlist = await wishlistService.addToWishlist( // (d) wishlist
    req.user.id, // C-use: req.user.id
    req.params.productId // C-use: req.params.productId
  );
});
```

```
res.status(201).json({ data: { wishlist } }); // C-use: wishlist
});
```

4. Analisis Jenis Data Anomali

Tiga jenis anomali aliran data yang diuji pada seluruh controller aplikasi BABEPUS:

Jenis Anomali	Penjelasan	Contoh Kasus JavaScript	Dampak pada Sistem
ur (use-before-define)	Variabel dibaca sebelum dideklarasikan	<code>console.log(token); const token = signToken(...);</code>	ReferenceError — aplikasi crash, request gagal
du (define-then-define)	Variabel diberi nilai baru tanpa membaca nilai sebelumnya	<code>let slug = 'lama'; slug = generateUniqueSlug(title);</code>	Dead assignment — nilai pertama terbuang, pemborosan komputasi
dk (define-then-kill)	Variabel diberi nilai lalu tidak pernah digunakan	<code>const conn = await pool.getConnection(); // tidak pernah dipakai</code>	Connection leak, pemborosan resource memori

4.1 Hasil Audit Statis Seluruh Controller

Berdasarkan penelusuran statis terhadap 13 file controller, tidak ditemukan anomali jenis ur, du, maupun dk. Temuan positif:

- Semua variabel result (data, user, product, offer, dsb.) selalu digunakan dalam `res.json()` setelah dideklarasikan — tidak ada dk.
- Tidak ada variabel yang digunakan sebelum dideklarasikan — tidak ada ur.
- Variabel `imageUrl` pada `productController` dan `userController` menggunakan ternary (`req.file ? ... : null`) — tidak ada du karena hanya satu assignment.
- SSE pattern (`streamChat`, `streamNotifications`) dengan baik mendaftarkan dan membersihkan listener via `req.on('close')` — tidak ada resource leak.

5. Evaluasi Akhir dan Kesimpulan

5.1 Ringkasan Status Aliran Variabel Global

Variabel	Sumber Definisi (d)	Lokasi Penggunaan (u)	Status
email	<code>req.body.email</code>	<code>pool.query WHERE email = ? (authService)</code>	Valid
password	<code>req.body.password</code>	<code>comparePassword(password, hash)</code>	Valid
token (JWT)	<code>signToken({ sub: user.id, role })</code>	<code>return { token, user } → response JSON</code>	Valid
productId	<code>req.params.id</code>	<code>assertProductOwner(), WHERE id</code>	Valid

Variabel	Sumber Definisi (d)	Lokasi Penggunaan (u)	Status
		= ? pada query	
imageUrl	getUploadUrl(req.file.filename)	createProduct() / updateProduct() / updateAvatar()	Valid
slug	generateUniqueSlug(title)	INSERT INTO products (... , slug, ...)	Valid
keepAlive (SSE)	setInterval(() => res.write('ping'), 25000)	clearInterval(keepAlive) saat req.on('close')	Valid
send (SSE fn)	const send = (data) => res.write(...)	.on(event, send) dan .off(event, send)	Valid
isSuspended	req.body.isSuspended	suspendUser(..., isSuspended) dan ternary message	Valid
note (dispute)	req.body.note	disputeEscrow(..., note) → INSERT dispute record	Valid

5.2 Kesimpulan

Berdasarkan hasil White Box Testing menggunakan metode Data Flow Testing terhadap 13 modul controller aplikasi BABEPUS, dapat disimpulkan:

7. Tidak ditemukan anomali aliran data jenis ur, du, maupun dk pada seluruh modul yang diuji.
8. Seluruh variabel kunci memiliki jalur definisi-penggunaan (DU-Path) yang valid dan bersih.
9. Implementasi keamanan data sudah diterapkan dengan benar: parameterized query mencegah SQL Injection, bcrypt mengamankan password, dan JWT mengamankan autentikasi.
10. Pola SSE (Server-Sent Events) pada modul Chat dan Notification telah mengimplementasikan pembersihan listener yang benar, mencegah memory leak.
11. Sistem mampu menjalankan seluruh 13 modul utama sesuai rancangan dengan aliran data yang stabil dan terstruktur.